K-Fold Cross Validation

K-Fold Cross Validation splits the dataset into *k* equal-sized folds. The model is trained on *k-1* folds and tested on the remaining fold. This process is repeated *k* times each time using a different fold for testing.
Note: *A commonly used value of k is 10, but the choice depends on the dataset size and problem requirements.*

Repeated K-Fold Cross Validation

This method repeats the K-Fold cross-validation process multiple times with different random splits. It helps reduce the effect of randomness in data splitting and provides a more robust performance estimate.

- The dataset is split into k folds multiple times.
- Each repetition uses a different random shuffle.
- Final performance is averaged across all repetitions.
- It is useful for improving reliability in model evaluation.

Example of K Fold Cross Validation

The diagram below shows an example of the training subsets and evaluation subsets generated in k-fold cross-validation. Here we have total 25 instances.

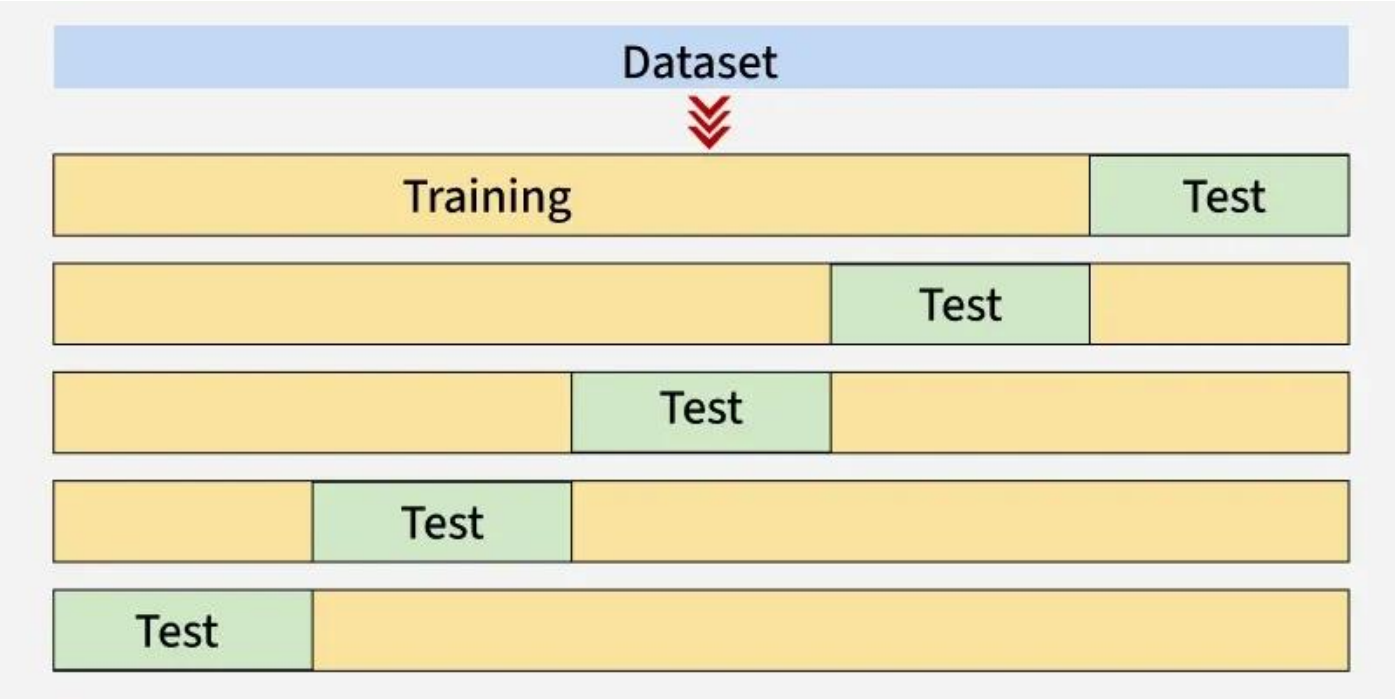


FIG : K Fold Cross Validation

- Here we will take k as 5.
- **1st iteration:** The first 20% of data [1–5] is used for testing and the remaining 80% [6–25] is used for training.

- **2nd iteration:** The second 20% [6–10] is used for testing and the remaining data [1–5] and [11–25] is used for training.
- This process continues until each fold has been used once as the test set.

Iteration	Training Set Observations	Testing Set Observations
1	[5-24]	[0-4]
2	[0-4, 10-24]	[5-9]
3	[0-9, 15-24]	[10-14]
4	[0-14, 20-24]	[15-19]
5	[0-19]	[20-24]

Each iteration uses different subsets for testing and training, ensuring that all data points are used for both training and testing.

Comparison between K-Fold Cross-Validation and Hold Out Method

K-Fold Cross-Validation and Hold Out Method are used technique and sometimes they are confusing so here is the quick comparison between them:

Feature	K-Fold Cross-Validation	Holdout Method
Data Split	Dataset is divided into k folds and each fold is used once as test set	Dataset is split once, typically into training and testing sets
Training & Testing	Model is trained and tested k times, each fold serving as test set once	Model is trained once on training set and tested once on test set
Bias & Variance	Lower bias, more reliable performance estimate and variance depends on k	Higher bias if the split is not representative and results can vary significantly
Execution Time	Slower, especially for large datasets because model is trained k times	Faster, only one training and testing cycle

Feature	K-Fold Cross-Validation	Holdout Method
Best Use Case	Small to medium datasets where accuracy estimation is important	Very large datasets or when quick evaluation is needed

Python implementation for k fold cross-validation

Step 1: Importing necessary libraries

We will import essential modules from scikit-learn.

- cross_val_score helps evaluate model performance using cross-validation.
- KFold splits the data into defined folds.
- SVC is used for Support Vector Classification.
- load_iris loads the sample dataset.

```
from sklearn.model_selection import cross_val_score, KFold
from sklearn.svm import SVC
from sklearn.datasets import load_iris
```

Step 2: Loading the dataset

We will use the Iris dataset a built-in, multi-class dataset with 150 samples and 3 flower species (Setosa, Versicolor and Virginica).

```
iris = load_iris()
X, y = iris.data, iris.target
```

Step 3: Creating SVM classifier

SVC() from scikit-learn is used to build the Support Vector Machine model. Here, we are using a linear kernel, suitable for linearly separable data.

```
svm_classifier = SVC(kernel='linear')
```

Step 4: Defining the number of folds for cross-validation

We define 5 folds, meaning the dataset will be split into 5 parts. The model will train on 4 parts and test on 1, repeating this process 5 times for balanced evaluation.

```
num_folds = 5
```

```
kf = KFold(n_splits=num_folds, shuffle=True, random_state=42)
```

Step 5: Performing k-fold cross-validation

We use cross_val_score() to automatically split data, train and evaluate the model across all folds. It returns the accuracy for each fold

```
cross_val_results = cross_val_score(svm_classifier, X, y, cv=kf)
```

Step 6: Evaluation metrics

We print individual fold accuracies and the mean accuracy across all folds to understand the model's stability and generalization.

```
print("Cross-Validation Results (Accuracy):")
for i, result in enumerate(cross_val_results, 1):
   print(f"  Fold {i}: {result * 100:.2f}%")

print(f'Mean Accuracy: {cross_val_results.mean()* 100:.2f}%')
```

Output:

```
Cross-Validation Results (Accuracy):
  Fold 1: 100.00%
  Fold 2: 100.00%
  Fold 3: 96.67%
  Fold 4: 93.33%
  Fold 5: 96.67%
Mean Accuracy: 97.33%
```

Cross validation accuracy

The output shows the accuracy scores from each of the 5 folds in the K-fold cross-validation process. The mean accuracy is the average of these individual scores which is approximately 97.33% indicating the model's overall performance across all the folds.

Advantages

1. **Better performance estimate:** Provides a more reliable evaluation than a single train-test split.
2. **Reduces overfitting:** Helps ensure the model generalizes well to unseen data.
3. **Efficient use of data:** All data points are used for both training and testing at different iterations.
4. **Flexible:** Works with different types of datasets and models.

Disadvantages

1. **Computationally Expensive:** It can be computationally expensive especially when the number of folds is large.
2. **Time-consuming:** Methods like LOOCV can take a long time for datasets with many data instances.
3. **Bias-Variance Tradeoff:** Few folds may result in high bias while too many folds may result in high variance.

3. Stratified Cross-Validation

It is a technique that ensures each fold of the cross-validation process has the same class distribution as the full dataset. This is useful for imbalanced datasets where some classes are underrepresented.

- The dataset is divided into k folds, keeping class proportions consistent in each fold.
- In each iteration, one fold is used for testing and the remaining folds for training.
- This process is repeated k times so that each fold is used once as the test set.
- It helps classification models generalize better by maintaining balanced class representation.

2. LOOCV (Leave One Out Cross Validation)

In this method the model is trained on the entire dataset except for one data point which is used for testing. This process is repeated for each data point in the dataset.

- All data points are used for training, resulting in low bias.
- Testing on a single data point can cause high variance, especially if the point is an outlier.
- It can be very time-consuming for large datasets as it requires one iteration per data point.